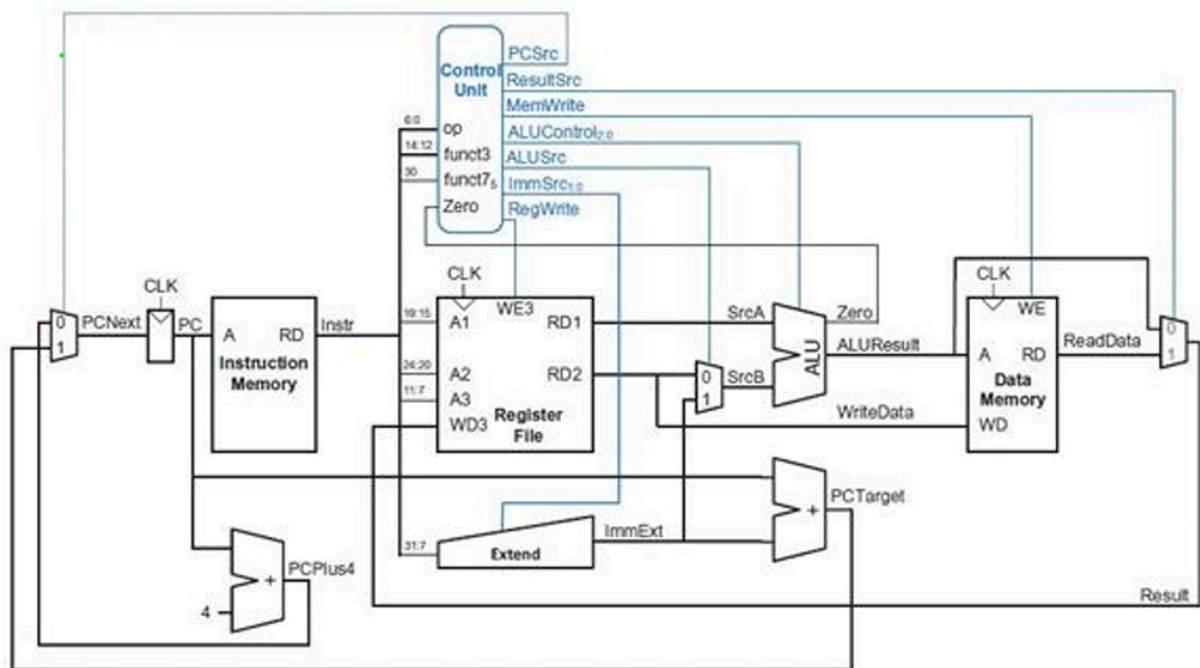


Single Cycle RISC Processor (RV32I)

Introduction

this project is a 32-bit single-cycle microarchitecture RISC-V processor based on Harvard Architecture. The single cycle microarchitecture executes an entire instruction in one cycle. In other words, instruction fetch, instruction decode, execute, write back, and program counter update occurs within a single clock cycle.

i was able to write the RTL Verilog files for all sub modules of the RISC-V processor (e.g. Register File, Instruction Memory, etc.). Then, implemented the top module of the RISC-V processor.

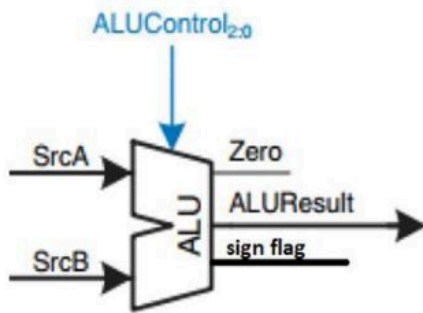


“Block diagram for the processor”

Processor Datapath & Submodules

1.ALU:

An Arithmetic/Logical Unit (ALU) combines a variety of mathematical and logical operations into a single unit.



ALU Control	Function
000	$A + B$
001	$A \ll B$
010	$A - B$
100	$A \text{ XOR } B$
101	$A \gg B$
110	$A \text{ OR } B$
111	$A \text{ AND } B$
Otherwise	0

Verilog code

```

module ALU (
    input [31:0] A,
    input [31:0] B,
    input [2:0] ALUControl,
    output reg [31:0] ALUOut,
    output reg Zero,
    output reg sign_flag
);

always @(*) begin

    case(ALUControl)
        3'b000 : ALUOut = A + B;           // Addition
        3'b001 : ALUOut = A << B;         // Shift left logical
        3'b010 : ALUOut = A - B;           // Subtraction
        3'b100 : ALUOut = A ^ B;           // XOR
        3'b101 : ALUOut = A >> B;         // Shift right logical
        3'b110 : ALUOut = A | B;           // OR
        3'b111 : ALUOut = A & B;           // AND
        default : ALUOut = 32'b0;          // Default case to prevent Latches
    endcase

    Zero = (ALUOut == 32'b0);
    sign_flag = ALUOut[31];
end

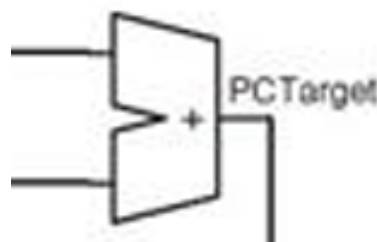
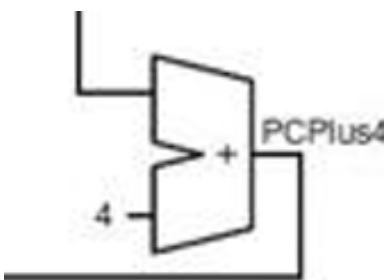
endmodule

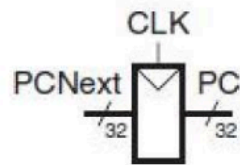
```

2. Program Counter:

To fetch the instructions from the instruction memory, we need a pointer to keep track of the address of the current instruction for this task we use the program counter.

- Program Counter Register.
- Next PC calculation logic.





areset	load	clk	PC
0	X	X	0
1	0	Posedge	PC
1	1	Posedge	PCNext
1	X	Not posedge	PC

PCSrc	PCNext
0	4 + PC
1	ImmExt + PC

Verilog code

```

module counter (
    input clk ,arst,load,
    input PCsrc,          // select between PC+4 and branch target
    input [31:0] immext,  // immediate value for branch target
    output reg [31:0] PC // program counter
);
    reg [31:0] PCnext;
    wire [31:0] PCtarget;
    wire [31:0] PCplus4;

    assign PCtarget = PC+immext;
    assign PCplus4  = PC+32'd4;

    always @(*) begin
        if (PCsrc)
            PCnext = PCtarget;
        else
            PCnext = PCplus4;
    end

    always @(posedge clk or negedge arst) begin
        if(!arst) PC <= 32'd0;

        else begin
            if(load)
                PC<=PCnext;
            else
                PC<=PC;
        end
    end
end
endmodule

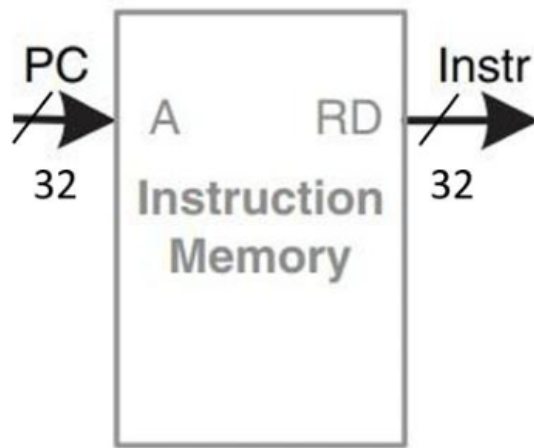
```

3. Instruction Memory:

The instruction memory is a Read Only Memory (ROM) that stores the program your CPU will execute.

ROM memory has width = 32 bits and depth = 64 entries.

NOTE: Instruction memory is word aligned (give it address [31:2]).



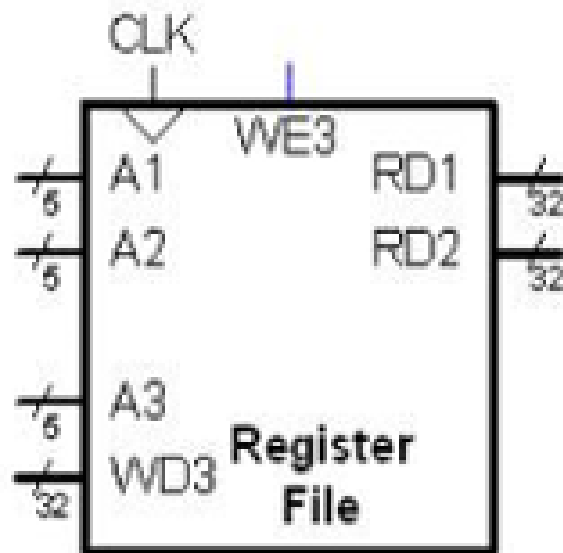
Verilog code

```
module instr_mem (  
    input [31:0] PC,  
    output reg [31:0] instruction  
);  
  
    reg [31:0] instruction_mem [0:63]; // 64 words of instruction memory  
  
    initial begin  
        $readmemh("instructions.txt", instruction_mem); // Load instructions from a file  
    end  
  
    always @(*) begin  
        instruction = instruction_mem[PC[31:2]]; // Word aligned [31:2]  
    end  
  
endmodule
```

4. Register File:

The Register File contains 32-bit registers.

It has two read output ports and one write input port.



Verilog code

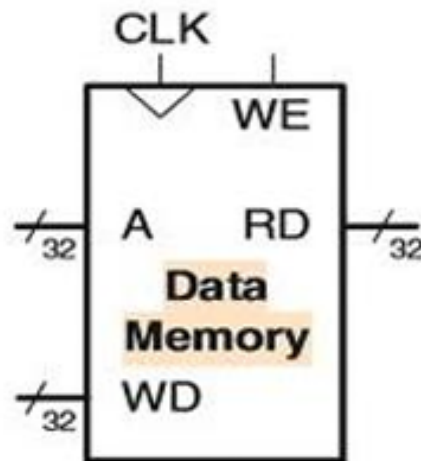
```
module register_file (
    input clk,rst,
    input [4:0] A1,A2,A3,
    input write_en,
    input [31:0] write_data,
    output reg [31:0] read_data1,read_data2
);
    reg [31:0] reg_file [0:31];
    always @(posedge clk or negedge rst) begin
        if(!rst) begin : RESET_BLOCK // Reset block to initialize the register file
            integer i;
            for (i = 0; i < 32; i = i + 1) begin
                reg_file[i] <= 32'b0;
            end
        end

        else if(write_en) begin
            if (A3 != 5'b0) begin // Prevent writing to register x0
                reg_file[A3] <= write_data;
            end
        end
    end

    always @(*) begin
        read_data1 = reg_file[A1];
        read_data2 = reg_file[A2];
    end
endmodule
```

5. Data Memory:

The data memory has a single read/write port.



Verilog code

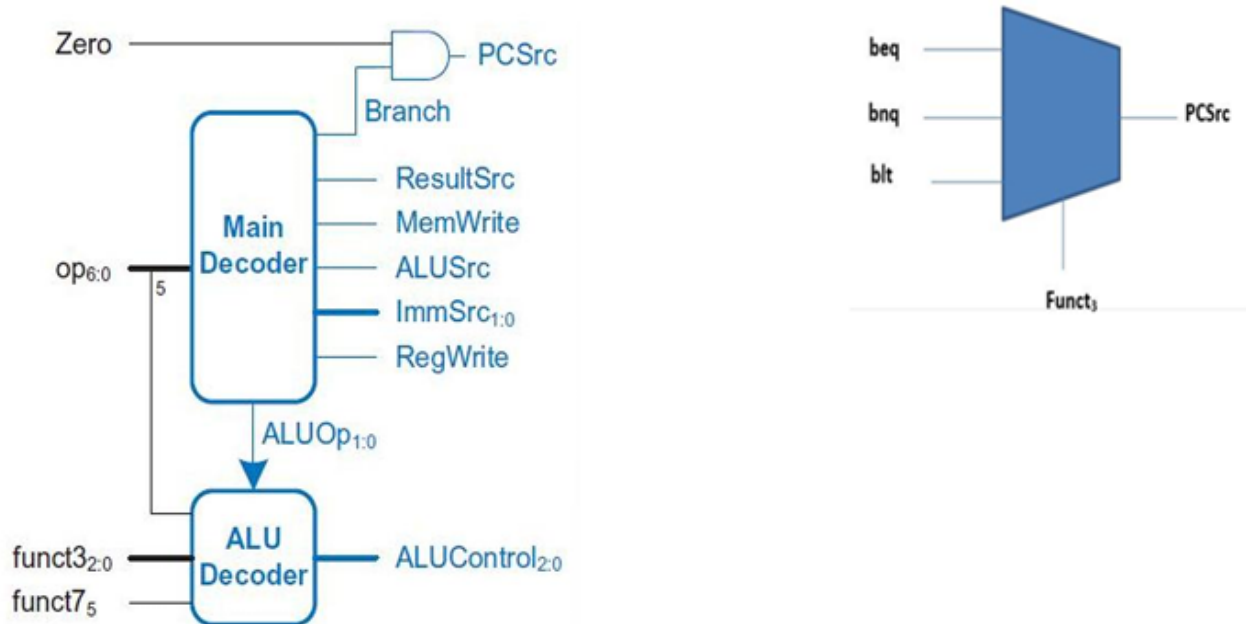
```
module Data_memory (
    input clk,rst,
    input [31:0] address,
    input [31:0] write_data,
    input write_en,
    output reg [31:0] read_data
);
    reg [31:0] data_mem [0:63];
    always @( posedge clk or negedge rst ) begin
        if(!rst) begin : RESET_BLOCK // Reset block to initialize the data memory
            integer i;
            for (i = 0; i < 64; i = i + 1) begin
                data_mem[i] <= 32'b0;
            end
        end
        else if(write_en) begin
            data_mem[address[7:2]] <= write_data; // Word aligned [7:2] for 64 words of memory
        end
    end

    always @(*) begin
        read_data = data_mem[address[7:2]]; // Word aligned [7:2] for 64 words of memory
    end
endmodule
```

6. Control Unit:

The control unit computes the control signals based on the opcode and funct3, funct7 fields of the instruction.

Thus, we will simplify our design by factoring the control unit into two blocks of combinational logic, as shown in the figure below.



- Main decoder Truth Table:

OpCode	RegWrite	ImmSrc 1:0	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp 1:0
loadWord =7'b000_0011	1	00	1	0	1	0	00
StoreWord =7'b010_0011	0	01	1	1	X	0	00
R-Type =7'b011_0011	1	XX	0	0	0	0	10
I-type =7'b001_0011	1	00	1	0	0	0	10
Branch- instruction =7'b110_0011	0	10	0	0	X	1	01
Default	0	00	0	0	0	0	00

- ALU decoder:

ALUOP	FUNCT3	{OP5,funct7}	ALU control	Instruction
00	XXX	XX	000 (add)	LW, SW
01	000	XX	010 (sub)	BEQ
	001	XX	010 (sub)	BNQ
	100	XX	010 (sub)	BLT
10	000	00, 01, 10	000 (add)	add
	000	11	010 (sub)	sub
	001	XX	001 (SHL)	SHL
	100	XX	100 (XOR)	XOR
	101	XX	101 (SHR)	SHR
	110	XX	110 (OR)	OR
	111	XX	111 (AND)	AND
Default	XXX	XX	000	---

Verilog code

```

module control_unit (
    input [6:0] opcode,
    input [2:0] func3,
    input  func7,      // one bit
    input zero_flag,   // from ALU
    input sign_flag,   // from ALU
    output reg PCsrc, resultsrc, mem_write, ALU_src, regwrite, load,
    output reg [1:0] immsrc,
    output reg [2:0] ALU_control
);

reg [1:0] ALU_op;
reg branch;

// 1. Main Decoder
always @(*) begin
    case (opcode)
        7'b0000011 : // LW (Load Word)
            begin
                regwrite = 1'b1;
                immsrc    = 2'b00;
                ALU_src    = 1'b1;
                mem_write  = 1'b0;
                resultsrc  = 1'b1;
                branch     = 1'b0;
                ALU_op     = 2'b00;
                load       = 1'b1;
            end
    end
end

```

```
7'b0100011 : // SW (Store Word)
```

```
begin
```

```
    regwrite  = 1'b0;  
    immsrc    = 2'b01;  
    ALU_src   = 1'b1;  
    mem_write = 1'b1;  
    resultsrc = 1'bx;  
    branch    = 1'b0;  
    ALU_op    = 2'b00;  
    load      = 1'b1;
```

```
end
```

```
7'b0110011 : // R-type
```

```
begin
```

```
    regwrite  = 1'b1;  
    immsrc    = 2'bx;  
    ALU_src   = 1'b0;  
    mem_write = 1'b0;  
    resultsrc = 1'b0;  
    branch    = 1'b0;  
    ALU_op    = 2'b10;  
    load      = 1'b1;
```

```
end
```

```

7'b0010011 : // I-type ALU
begin
    regwrite   = 1'b1;
    immsrc     = 2'b00;
    ALU_src    = 1'b1;
    mem_write  = 1'b0;
    resultsrc  = 1'b0;
    branch     = 1'b0;
    ALU_op     = 2'b10;
    load       = 1'b1;
end

7'b1100011 : // B-type (Branch)
begin
    regwrite   = 1'b0;
    immsrc     = 2'b10;
    ALU_src    = 1'b0;
    mem_write  = 1'b0;
    resultsrc  = 1'bx;
    branch     = 1'b1;
    ALU_op     = 2'b01;
    load       = 1'b1;
end

```

```

default: // Default / HLT State
begin
    regwrite   = 1'b0;
    immsrc     = 2'b00;
    ALU_src    = 1'b0;
    mem_write  = 1'b0;
    resultsrc  = 1'b0;
    branch     = 1'b0;
    ALU_op     = 2'b00;
    load       = 1'b0;
end
endcase
end

```

```

// 2. ALU Decoder
always @(*) begin
  case (ALU_op)
    2'b00: ALU_control = 3'b000; // ADD
    2'b01: ALU_control = 3'b010; // SUB

    2'b10: begin
      if (func3 == 3'b000) begin
        if (opcode[5] && func7)
          ALU_control = 3'b010; // SUB
        else
          ALU_control = 3'b000; // ADD
      end
      else begin
        ALU_control = func3;
      end
    end

    default: ALU_control = 3'b000;
  endcase
end

```

7. Sign extend:

Sign extension simply copies the sign bit (most significant bit) of a short input (16 bits) into all the upper bits of the longer output (32 bits).

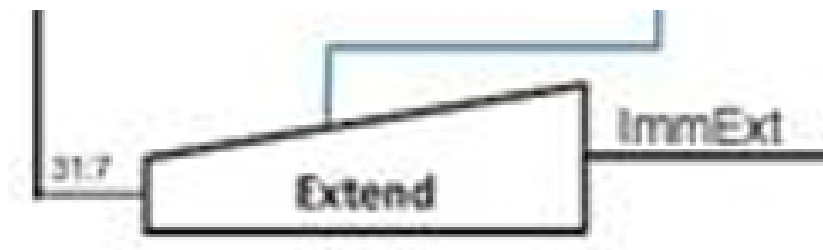
→ example

- Inst [15:0] = 0000 0000 0011 1101
ImmExt = 0000 0000 0000 0000 0000 0000 0011 1101
- Inst [15:0] = 1000 0000 0011 1101

Table 7.1 ImmSrc encoding

ImmSrc	ImmExt	Type	Description
00	{{20{Instr[31]}}, Instr[31:20]}	I	12-bit signed immediate
01	{{20{Instr[31]}}, Instr[31:25], Instr[11:7]}	S	12-bit signed immediate
10	{{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0}	B	13-bit signed immediate

ImmExt = 1111 1111 1111 1111 1000 0000 0011 1101



Verilog code

```

module sign_extend (
    input [31:0] instruction,
    input [1:0] immsrc,
    output reg [31:0] immext
);
    always @(*) begin
        case (immsrc) // Determine the type of immediate based on immsrc
            2'b00: begin
                immext = { {20{instruction[31]}}, instruction[31:20] };
            end
            2'b01: begin
                immext = { {20{instruction[31]}}, instruction[31:25], instruction[11:7] };
            end
            2'b10: begin
                immext = { {20{instruction[31]}}, instruction[7], instruction[30:25], instruction[11:8], 1'b0 };
            end
            default: begin
                immext = 32'b0;
            end
        endcase
    end
endmodule

```

8.Top module

connecting all the submodules together

Verilog code

```

module top_processor (
    input clk, arst,
    output [31:0] PC,           // Program Counter
    output [31:0] ALUResult,    // Result from ALU
    output [31:0] Instruction,  // Instruction from Instruction Memory
    output [31:0] WriteData,    // Data to be written to Data Memory (ReadData2)
    output      MemWrite,       // enable signal for Data Memory write operation
    output [31:0] ReadData      // Data read from Data Memory
);

wire [6:0] opcode;
wire [2:0] func3;
wire      func7;
wire      zero_flag;
wire      sign_flag;
wire      PCsrc, resultsrc, ALU_src, regwrite, load;
wire [1:0] immsrc;
wire [2:0] ALU_control;
wire [31:0] PCnext, PCtarget;
wire [31:0] immext;
wire [31:0] read_data1;

// Muxes
wire [31:0] srcB;
wire [31:0] result;

```



```
// 1. Control Unit
control_unit cu(
    .opcode(Instruction[6:0]),
    .func3(Instruction[14:12]),
    .func7(Instruction[30]),
    .zero_flag(zero_flag),
    .sign_flag(sign_flag),
    .PCsrc(PCsrc),
    .resultsrc(resultsrc),
    .mem_write(MemWrite),
    .ALU_src(ALU_src),
    .regwrite(regwrite),
    .load(load),
    .immsrc(immsrc),
    .ALU_control(ALU_control)
);

// 2. Instruction Memory
instr_mem imem(
    .PC(PC),
    .instruction(Instruction)
);
```

```
// 3. Program Counter
```

```
counter pc_counter(  
    .clk(clk),  
    .arst(arst),  
    .load(load),  
    .PCsrc(PCsrc),  
    .immext(immext),  
    .PC(PC)  
);
```

```
// 4. Sign Extend
```

```
sign_extend sx(  
    .instruction(Instruction),  
    .immsrc(immsrc),  
    .immext(immext)  
);
```

```
// 5. Multiplexers
```

```
assign srcB    = (ALU_src == 1'b1) ? immext    : WriteData;  
assign result  = (resultsrc == 1'b1) ? ReadData : ALUResult;
```

```
// 6. Register File
```

```
register_file rf(  
    .clk(clk),  
    .rst(arst),  
    .write_en(regwrite),  
    .A1(Instruction[19:15]),  
    .A2(Instruction[24:20]),  
    .A3(Instruction[11:7]),  
    .write_data(result),  
    .read_data1(read_data1),  
    .read_data2(WriteData)  
);
```

```
// 7. ALU
```

```
ALU alu(  
    .A(read_data1),  
    .B(srcB),  
    .ALUControl(ALU_control),  
    .Zero(zero_flag),  
    .sign_flag(sign_flag),  
    .ALUOut(ALUResult)  
);
```

```
// 8. Data Memory
Data_memory dmem(
    .clk(clk),
    .rst(arst),
    .address(ALUResult),
    .write_data(WriteData),
    .write_en(MemWrite),
    .read_data(ReadData)
);

endmodule
```

Results and simulations:

Testbench code

```

module tb_top_processor;
    reg clk;
    reg arst;
    wire [31:0] PC;           // Program Counter
    wire [31:0] ALUResult;    // Result from ALU
    wire [31:0] Instruction;  // Instruction from Instruction Memory
    wire [31:0] WriteData;    // Data to be written to Data Memory
    wire        MemWrite;     // Enable signal for Data Memory
    wire [31:0] ReadData;     // Data read from Data Memory

    // 2. (Instantiation)
    top_processor uut (
        .clk(clk),
        .arst(arst),
        .PC(PC),
        .ALUResult(ALUResult),
        .Instruction(Instruction),
        .WriteData(WriteData),
        .MemWrite(MemWrite),
        .ReadData(ReadData)
    );

```

```

initial begin //clock generation
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    arst = 0; // reset the processor
    #12;
    arst = 1;

    $monitor("Time=%0t | PC=%h | Instr=%h | ALU_Out=%h | MemWrite=%b | WriteData=%h | ReadData=%h",
        $time, PC, Instruction, ALUResult, MemWrite, WriteData, ReadData);

    #800;
    $finish;
end

endmodule

```

run.do file

```
quit -sim

vlib work

vlog *.v

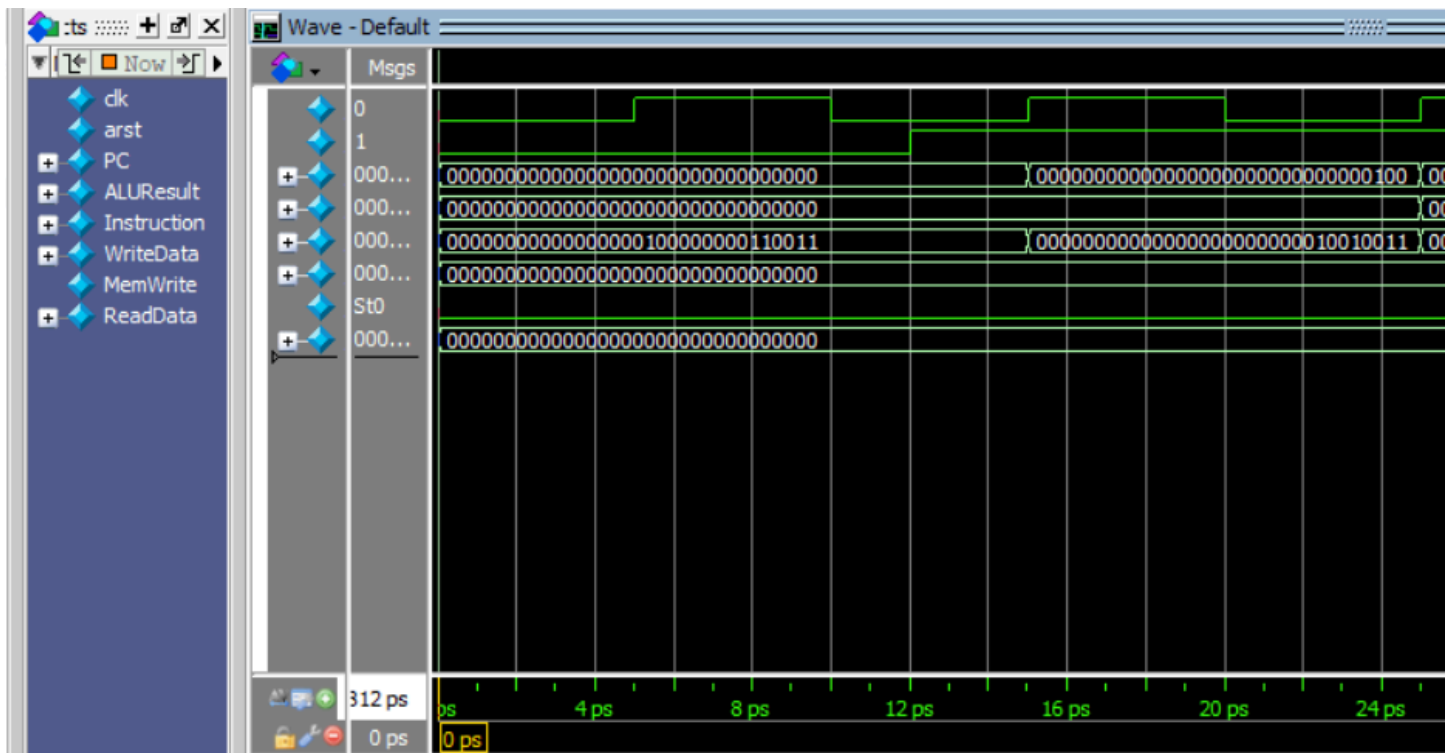
vsim work.tb_top_processor

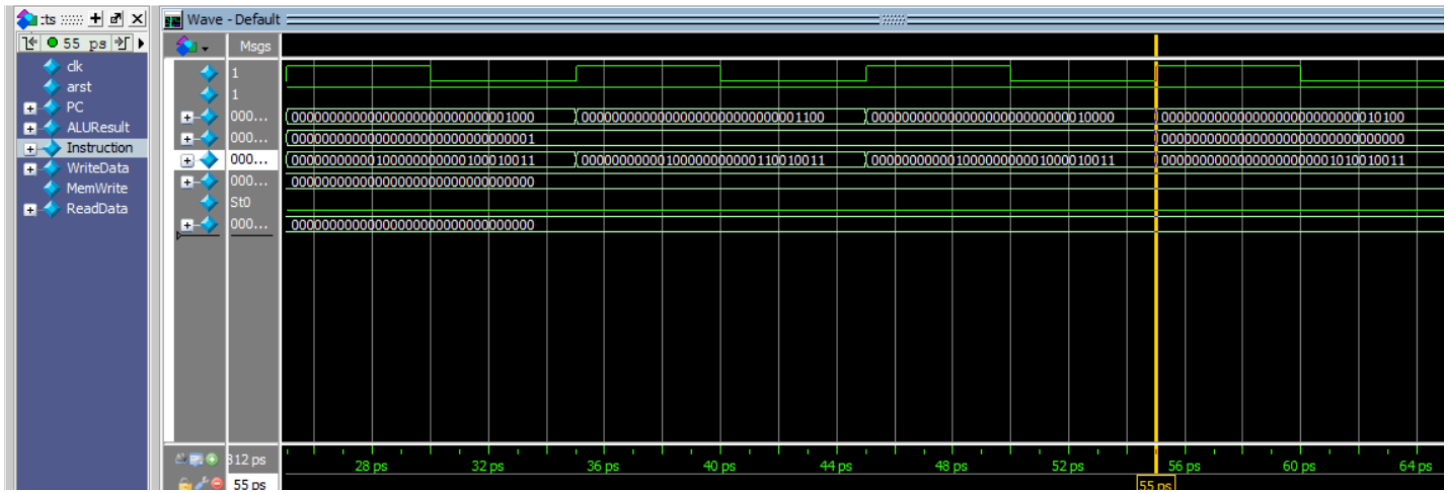
add wave -position insertpoint sim:/tb_top_processor/*

run -all

wave zoom full
```

waveform snippets





Program Counter Verification: As demonstrated in the simulation waveforms, the Program Counter (PC) increments sequentially by **4 bytes** per clock cycle (progressing smoothly from 0 to 4, 8, 12, 16, and 20 in hex), which perfectly validates the correct sequential execution of the instructions loaded from the memory file.

Control & Execution Paths: The control signals (such as MemWrite and RegWrite) toggle dynamically according to the current instruction decoded from the Instruction bus. The ALUResult updates precisely within the same clock cycle, confirming that the arithmetic, logic, and branch datapaths are executing simultaneously without any unexpected propagation delays or latches.

Conclusion

This project successfully demonstrates the comprehensive logic design and functional verification of a **32-bit Single-Cycle RISC-V (RV32I) Processor**. By modularly engineering each submodule from the instruction fetch logic to the execution and control datapaths the entire architecture achieves complete hardware determinism, executing one full instruction per clock cycle. The flawless alignment between the expected assembly behaviors, the RTL Verilog implementation, and the automated ModelSim simulation results confirms the robustness of the core architecture. This project serves as a solid foundational gateway toward exploring more advanced computer architecture paradigms, such as multi-cycle execution, pipelining, and superscalar designs.